Module 5: Introduction to Git

Aurora DesRosiers

IFT 458: Middleware Programming and Database Security

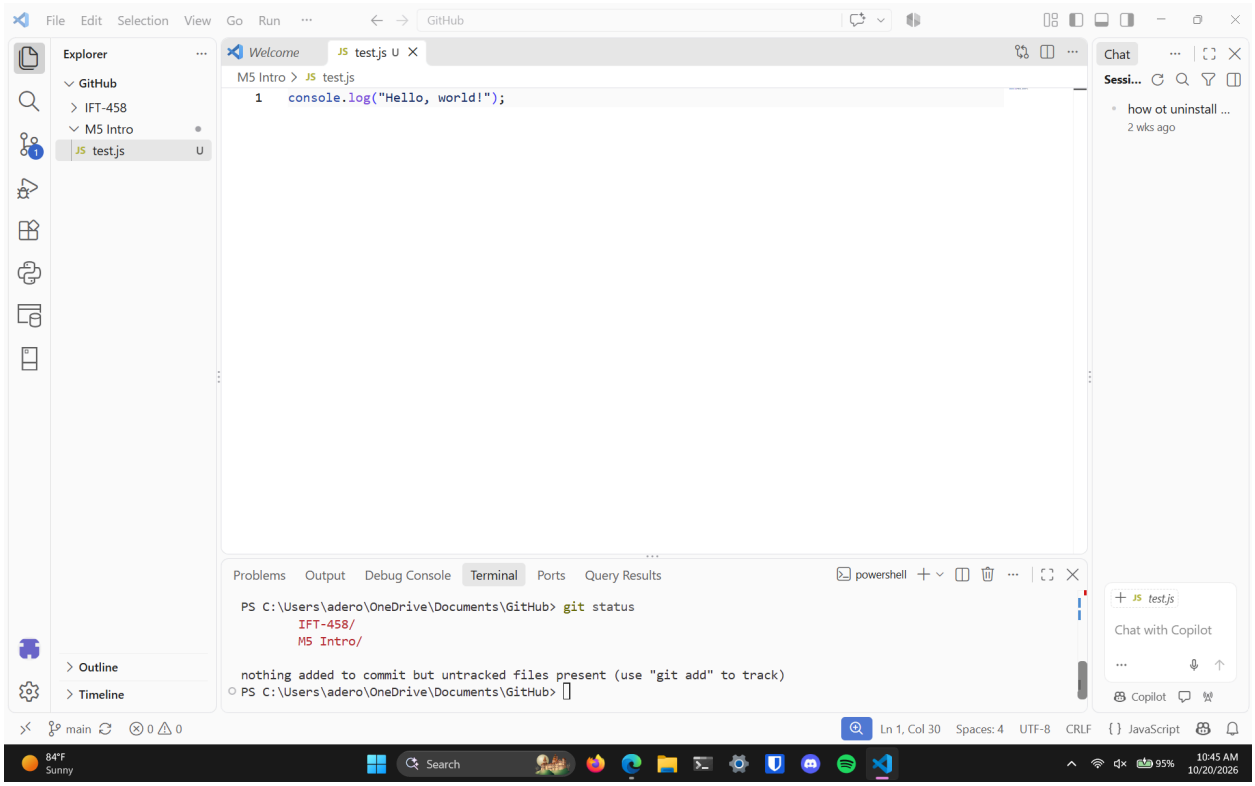
Professor Sthapit

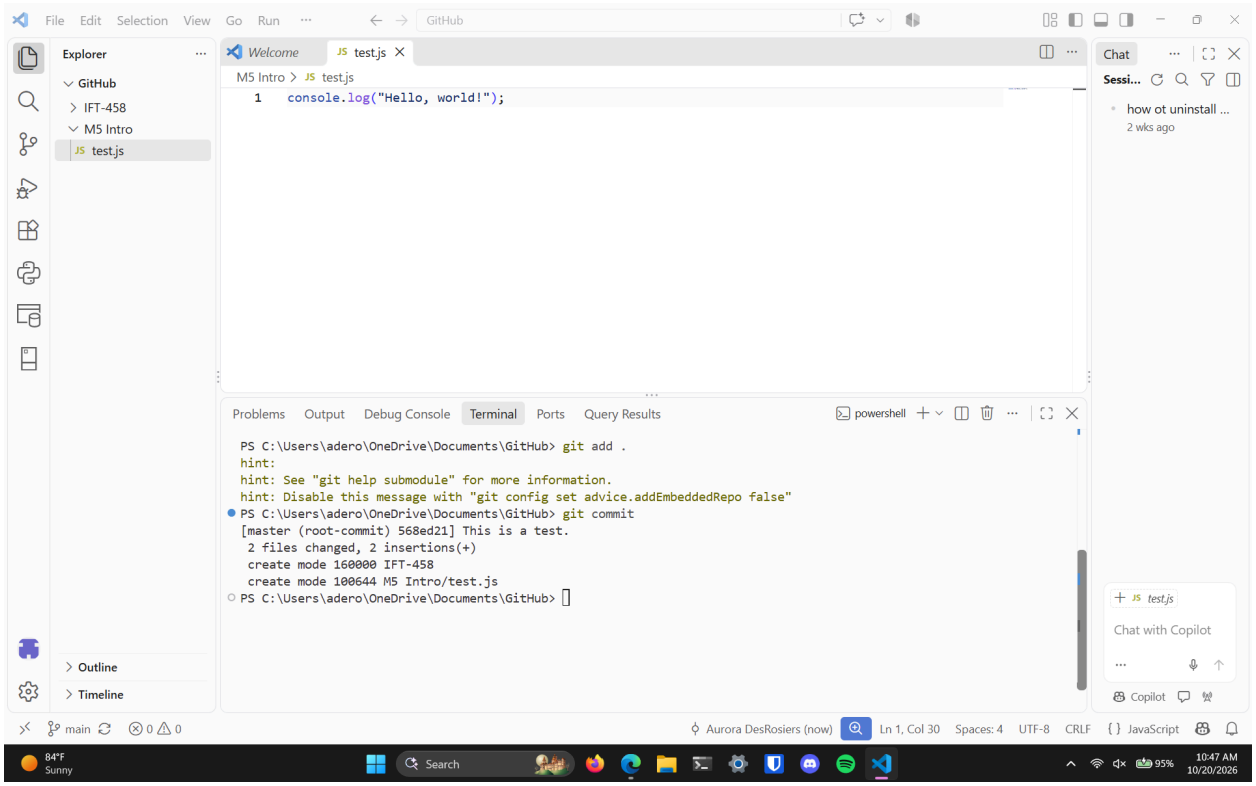
October 25, 2026

Module 5: Introduction to Git

1. Git files in a repository can be modified, staged, or committed. You could also argue the stages are unmodified, modified, and staged, but the areas go better with the first answer. The areas would be the Working Directory, the Staging Area, and the Repository. “Git add” is responsible for changing it from the Working Directory to the Staging Area, while the “git commit” command is responsible for changing it from the Staging Area to the Repository.

Figure 1: Three States



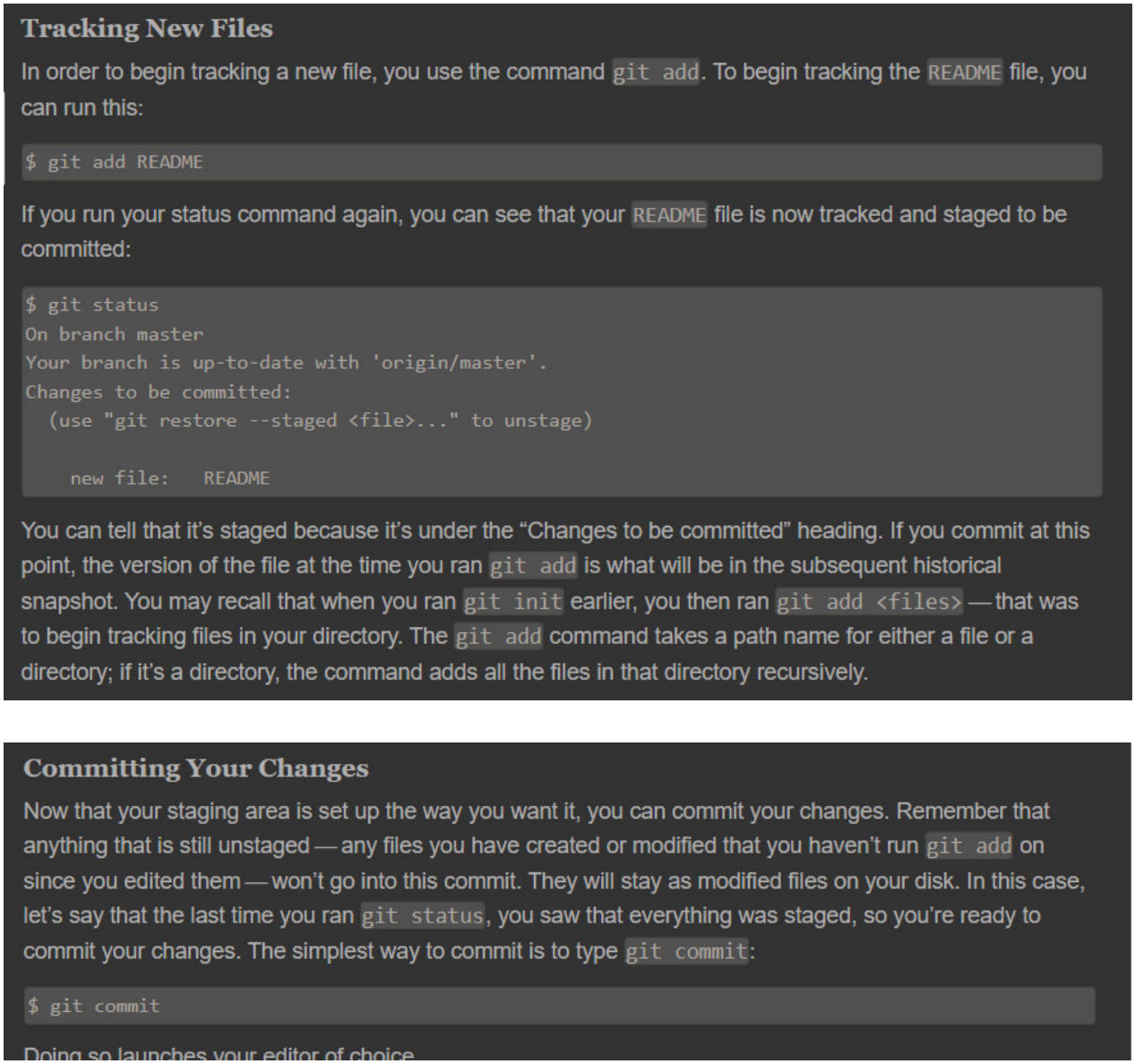


Checking the Status of Your Files

The main tool you use to determine which files are in which state is the `git status` command. If you run this command directly after a clone, you should see something like this:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working tree clean
```

This means you have a clean working directory; in other words, none of your tracked files are modified. Git also doesn't see any untracked files, or they would be listed here. Finally, the command tells you which branch you're on and informs you that it has not diverged from the same branch on the server. For now, that branch is always `master`, which is the default; you won't worry about it here. Git Branching will go over branches and references in detail.



Tracking New Files

In order to begin tracking a new file, you use the command `git add`. To begin tracking the `README` file, you can run this:

```
$ git add README
```

If you run your status command again, you can see that your `README` file is now tracked and staged to be committed:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)

    new file:   README
```

You can tell that it's staged because it's under the "Changes to be committed" heading. If you commit at this point, the version of the file at the time you ran `git add` is what will be in the subsequent historical snapshot. You may recall that when you ran `git init` earlier, you then ran `git add <files>` — that was to begin tracking files in your directory. The `git add` command takes a path name for either a file or a directory; if it's a directory, the command adds all the files in that directory recursively.

Committing Your Changes

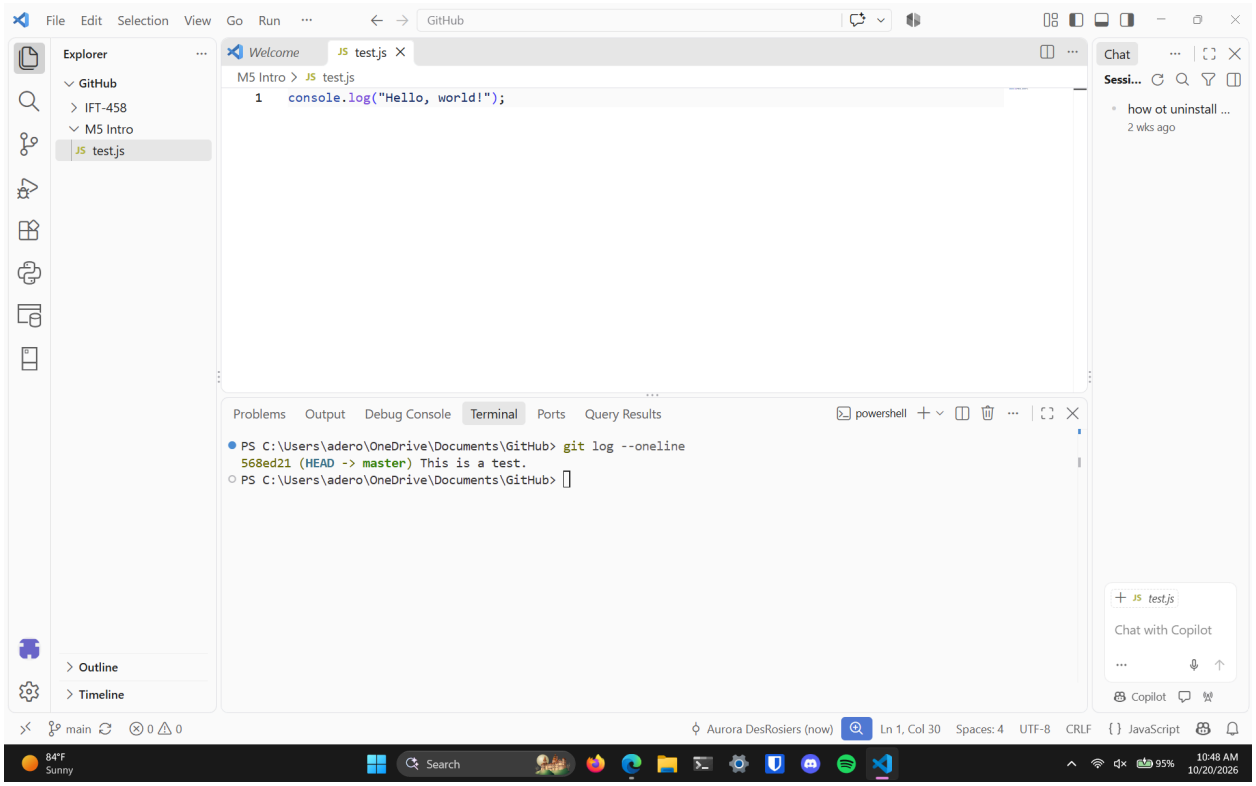
Now that your staging area is set up the way you want it, you can commit your changes. Remember that anything that is still unstaged — any files you have created or modified that you haven't run `git add` on since you edited them — won't go into this commit. They will stay as modified files on your disk. In this case, let's say that the last time you ran `git status`, you saw that everything was staged, so you're ready to commit your changes. The simplest way to commit is to type `git commit`:

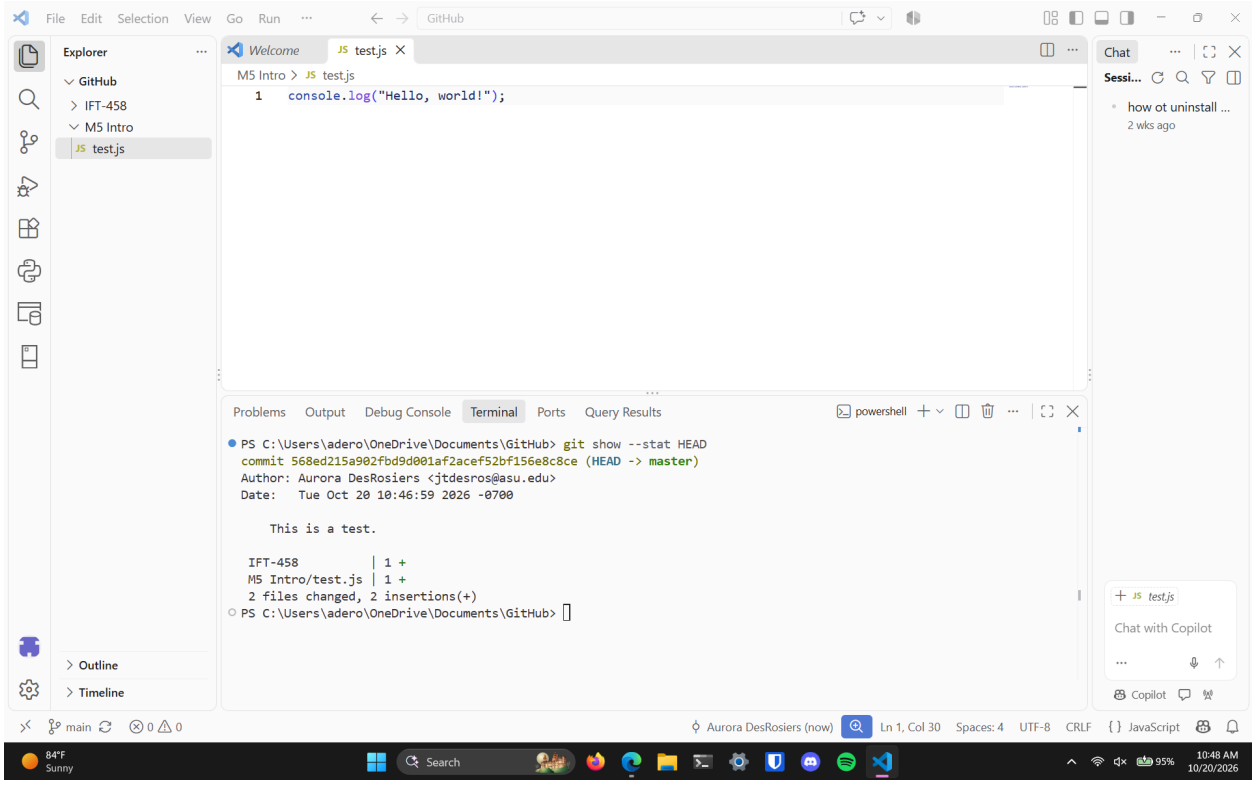
```
$ git commit
```

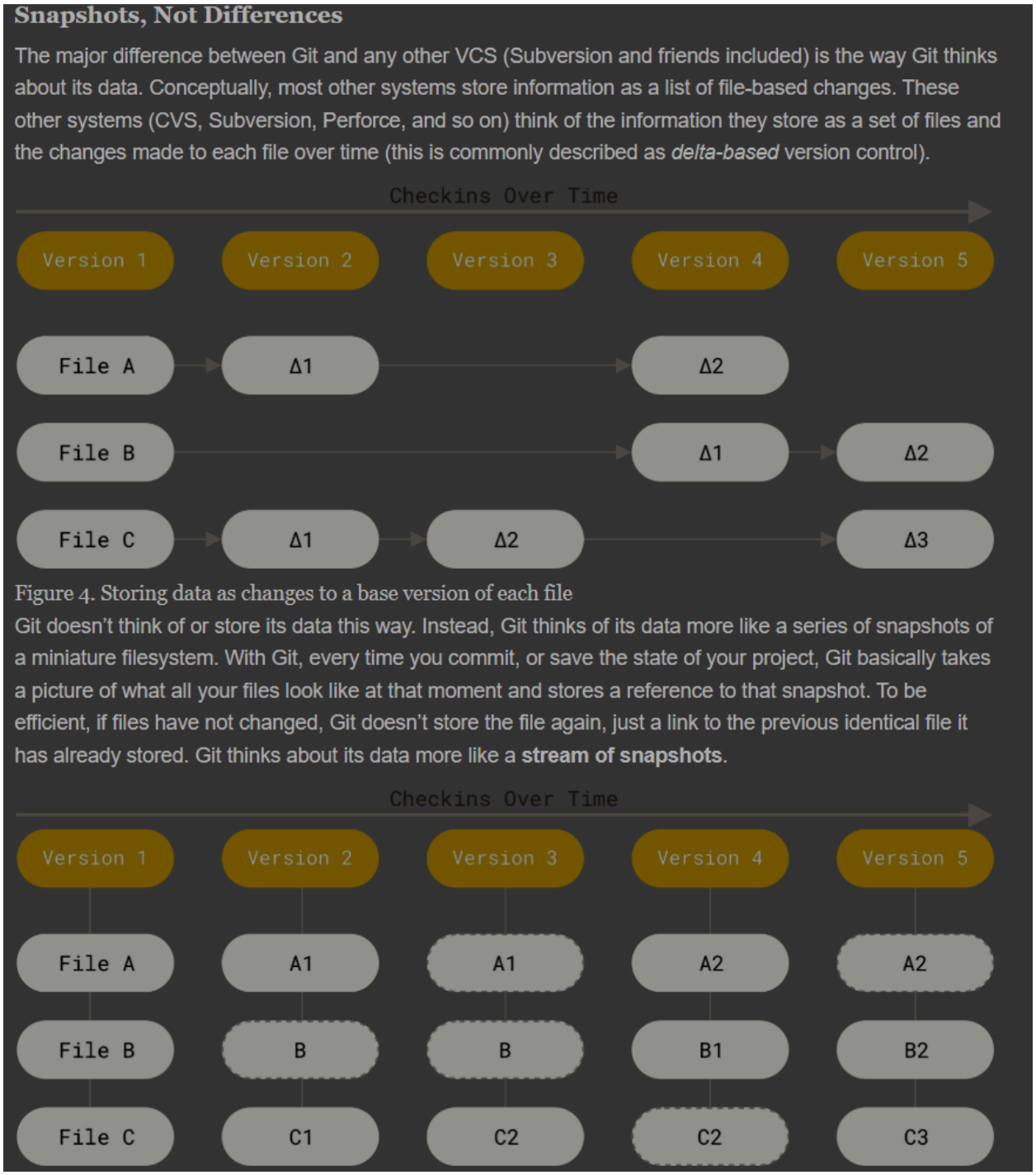
Doing so launches your editor of choice.

2. Git stores data with snapshots of files when you commit. When it hasn't been changed between commits, it just stores links to the file's last commit.

Figure 2: Snapshots Versus Differences


File Edit Selection View Go Run
GitHub
Explorer
GitHub
IFT-458
M5 Intro
test.js
Welcome
test.js
M5 Intro > test.js
1 console.log("Hello, world!");
Chat
Sessi...
how ot uninstall ...
2 wks ago
Problems Output Debug Console Terminal Ports Query Results
powershell
PS C:\Users\adero\OneDrive\Documents\GitHub> git log --oneline
568ed21 (HEAD -> master) This is a test.
PS C:\Users\adero\OneDrive\Documents\GitHub>
Outline
Timeline
test.js
Chat with Copilot
Copilot
main
Aurora DesRosiers (now)
Ln 1, Col 30
Spaces: 4
UTF-8
CRLF
JavaScript
84°F
Sunny
Search
95%
10:48 AM
10/20/2026

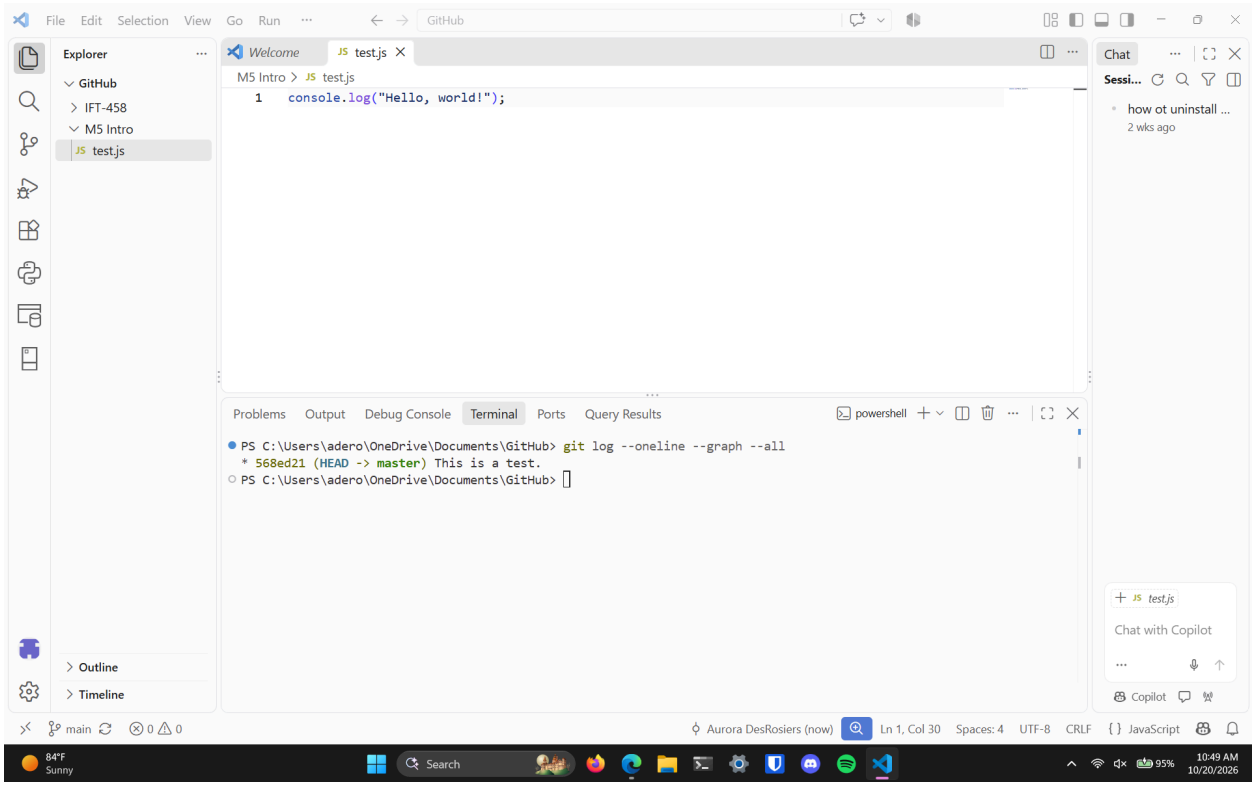

File Edit Selection View Go Run
GitHub
Explorer
GitHub
IFT-458
M5 Intro
test.js
Welcome
test.js
M5 Intro > test.js
1 console.log("Hello, world!");
Chat
Sessi...
how ot uninstall ...
2 wks ago
Problems Output Debug Console Terminal Ports Query Results
powershell
PS C:\Users\adero\OneDrive\Documents\GitHub> git show --stat HEAD
commit 568ed215a902fbd9d001af2acef52bf156e8c8ce (HEAD -> master)
Author: Aurora DesRosiers <jtdesros@asu.edu>
Date: Tue Oct 20 10:46:59 2026 -0700
This is a test.
IFT-458 | 1 +
M5 Intro/test.js | 1 +
2 files changed, 2 insertions(+)
PS C:\Users\adero\OneDrive\Documents\GitHub>
Outline
Timeline
test.js
Chat with Copilot
Copilot
main
Aurora DesRosiers (now)
Ln 1, Col 30
Spaces: 4
UTF-8
CRLF
JavaScript
84°F
Sunny
Search
95%
10:48 AM
10/20/2026



3. A branch is a pointer to a specific commit, HEAD is a special pointer that tells Git what branch you are currently on. The difference between "git merge" and "git rebase" when you integrate a feature branch into main is that "git merge" preserves history and adds a new mixed branch, while "git rebase" essentially overwrites and erases your history so it

looks cleaner. You should not rebase on public or an already shared repository, or else you could mess up work that other people are potentially doing.

Figure 3: Branching



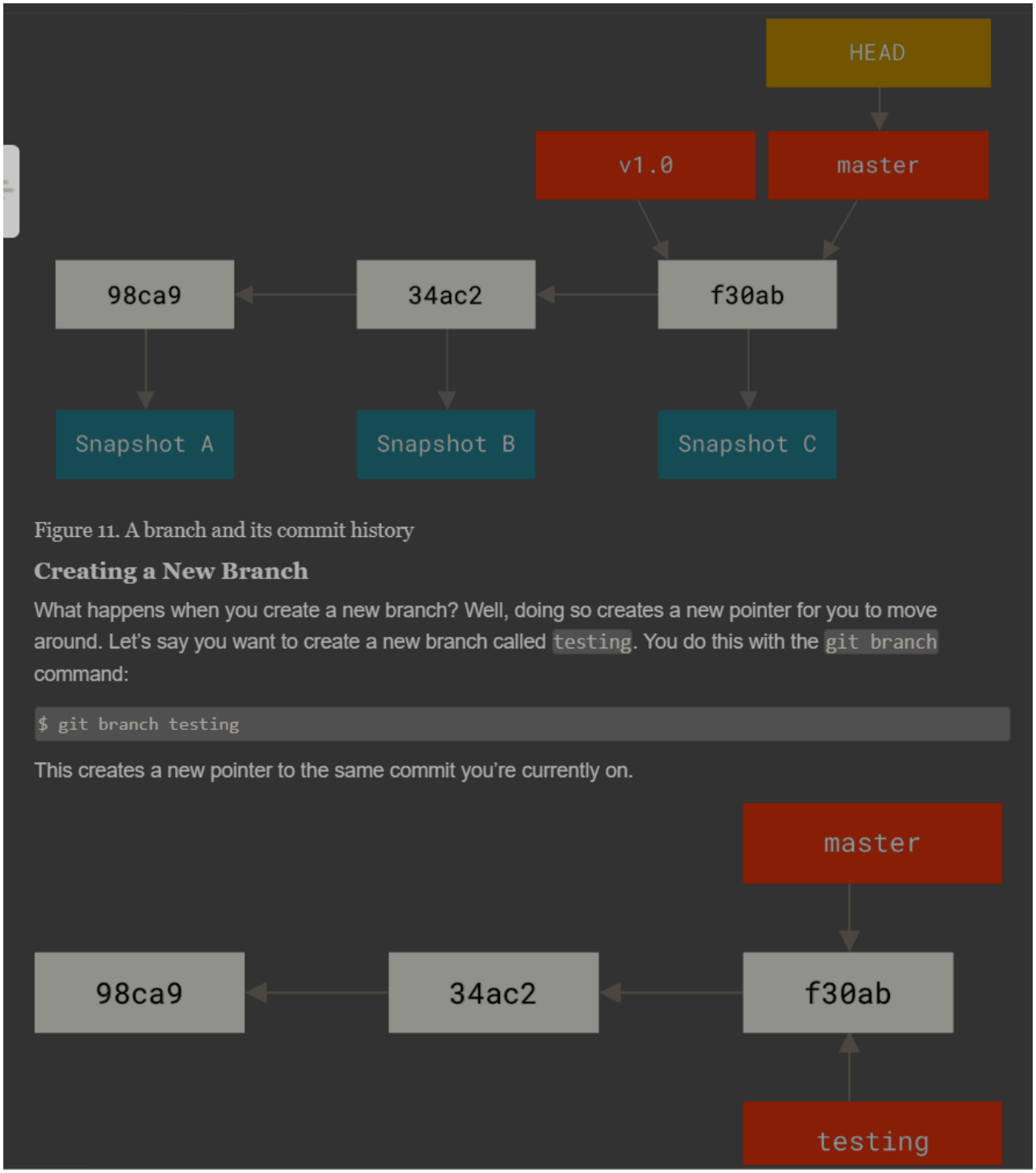


Figure 11. A branch and its commit history

Creating a New Branch

What happens when you create a new branch? Well, doing so creates a new pointer for you to move around. Let's say you want to create a new branch called `testing`. You do this with the `git branch` command:

```
$ git branch testing
```

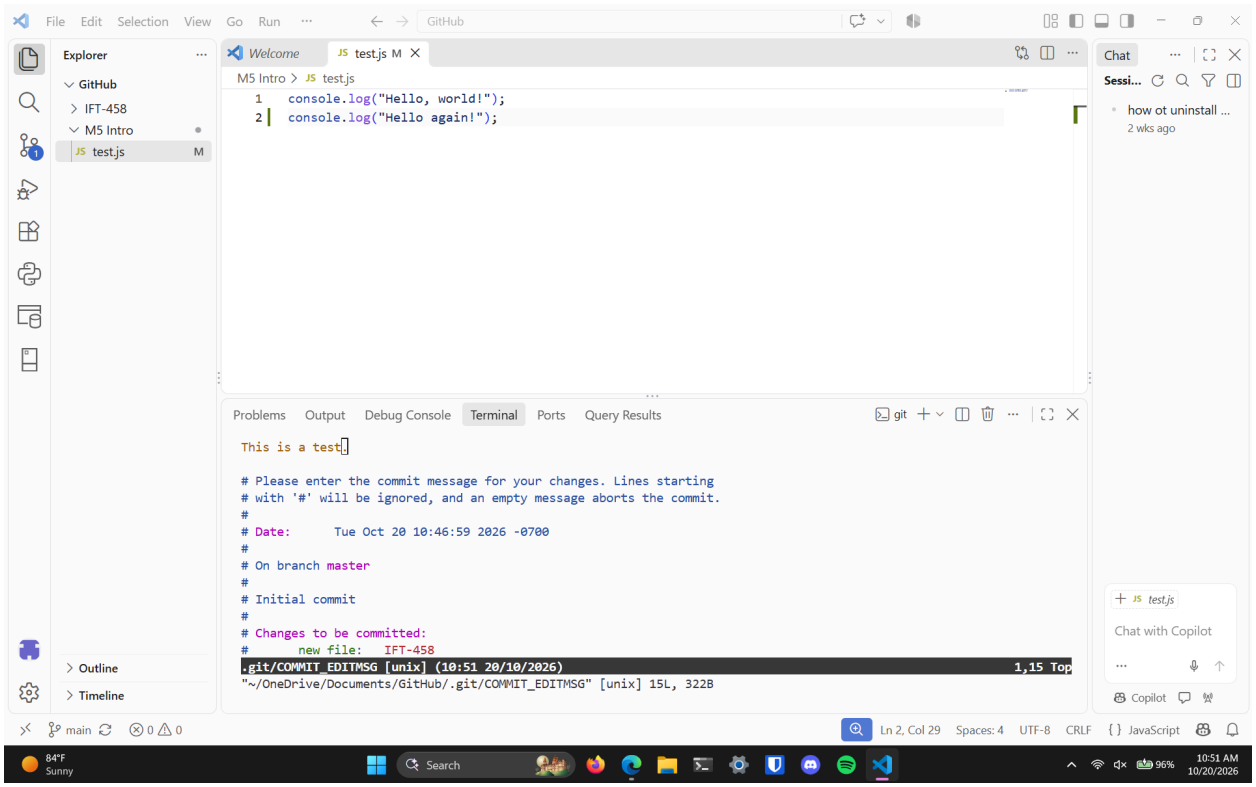
This creates a new pointer to the same commit you're currently on.

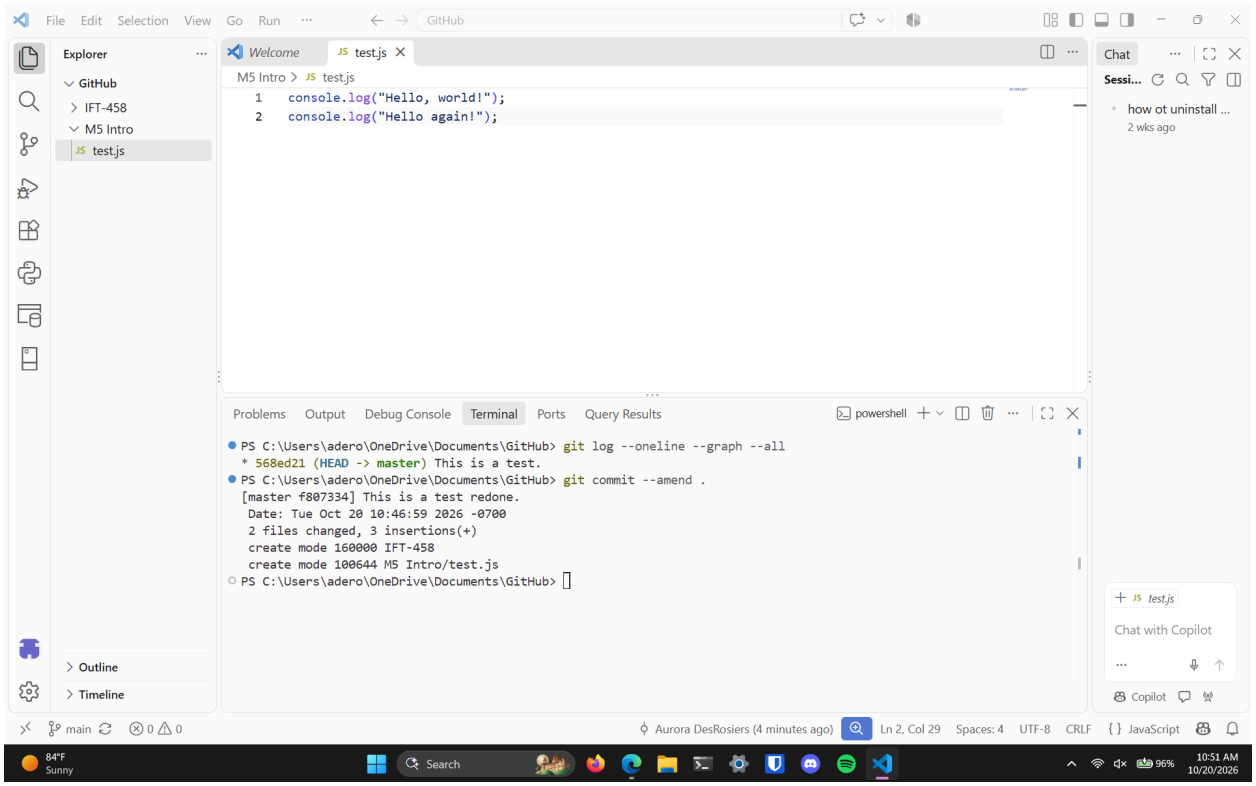
4. A) This is fixed by "git commit –amend".

   B) This is done by "git reset HEAD (file)" (it also looks like "git restore –staged (file)" does something similar).

C) The difference is that –soft keeps everything preserved except your HEAD which is moved. –mixed is when your HEAD and changes are unstaged or changed but your working directory is untouched. –hard resets everything and discards all changes. I found this using GeeksForGeeks as it was not in the Git Pro Book.

Figure 4: Undoing Things





Undoing Things

At any stage, you may want to undo something. Here, we'll review a few basic tools for undoing changes that you've made. Be careful, because you can't always undo some of these undos. This is one of the few areas in Git where you may lose some work if you do it wrong.

One of the common undos takes place when you commit too early and possibly forget to add some files, or you mess up your commit message. If you want to redo that commit, make the additional changes you forgot, stage them, and commit again using the `--amend` option:

```
$ git commit --amend
```

undo changes to them. For example, let's say you've changed two files and want to commit them as two separate changes, but you accidentally type `git add *` and stage them both. How can you unstage one of the two? The `git status` command reminds you:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
    modified:   CONTRIBUTING.md
```

Right below the "Changes to be committed" text, it says use `git reset HEAD <file>…` to unstage. So, let's use that advice to unstage the `CONTRIBUTING.md` file:

```
$ git reset HEAD CONTRIBUTING.md
Unstaged changes after reset:
M       CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README

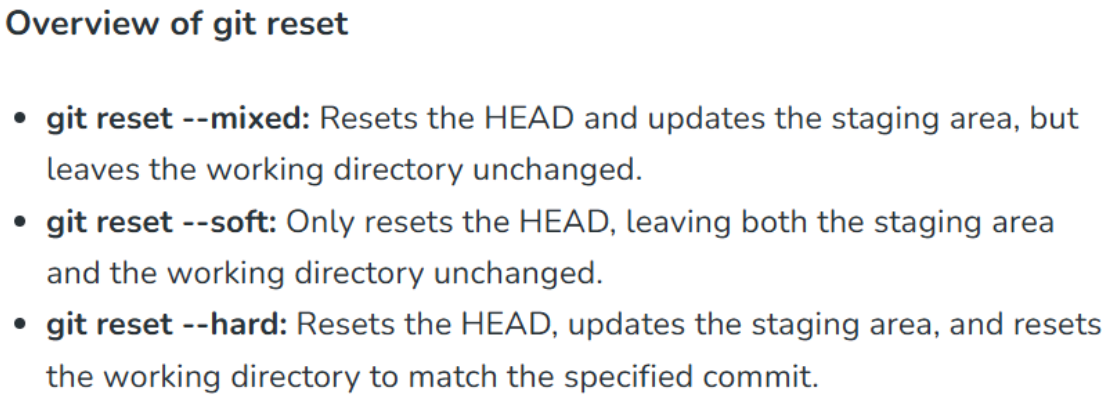
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

The command is a bit strange, but it works. The `CONTRIBUTING.md` file is modified but once again unstaged.

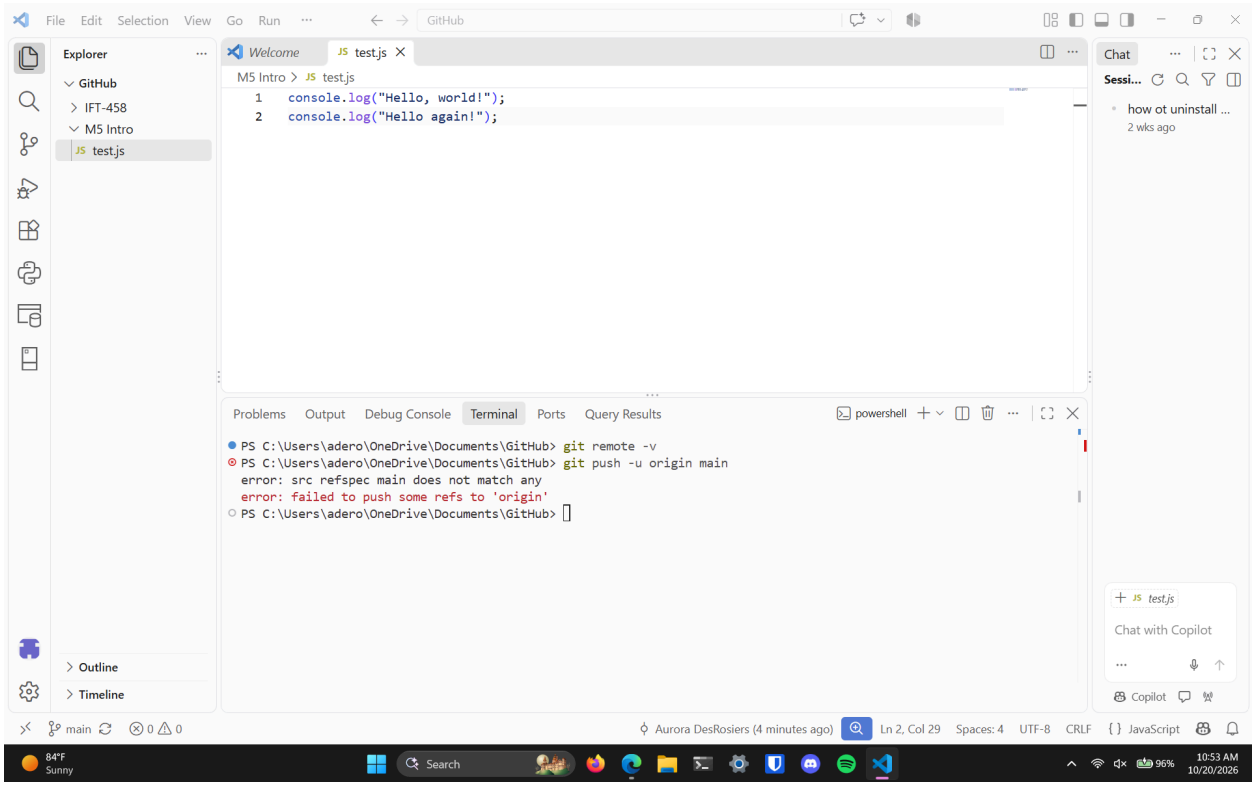
Note	It's true that `git reset` can be a dangerous command, especially if you provide the `--hard` flag. However, in the scenario described above, the file in your working directory is not touched, so it's relatively safe.

For now this magic invocation is all you need to know about the `git reset` command. We'll go into much more detail about what `reset` does and how to master it to do really interesting things in Reset Demystified.



5. The “git fetch” command downloads your remote repository, but does not update your working branch at all. The “git pull” command downloads the remote repository and pushes it into your working branch. “Git push” can be rejected due to someone else pushing before you, and you having a not up-to-date workspace. This can be fixed by fetching and merging into the most up-to-date work branch then trying again. The difference between “git clone” and “git init” is “git clone” clones an existing repository while “git init” simply creates a new one.

Figure 5: Remotes and Collaboration


File Edit Selection View Go Run
GitHub
Explorer
GitHub
IFT-458
M5 Intro
test.js
Welcome
test.js
M5 Intro > test.js
1 console.log("Hello, world!");
2 console.log("Hello again!");
Chat
how ot uninstall ...
2 wks ago
Problems Output Debug Console Terminal Ports Query Results
powershell
PS C:\Users\adero\OneDrive\Documents\GitHub> git remote -v
PS C:\Users\adero\OneDrive\Documents\GitHub> git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'origin'
PS C:\Users\adero\OneDrive\Documents\GitHub>
Outline
Timeline
Chat with Copilot
Copilot
main
Aurora DesRosiers (4 minutes ago)
Ln 2, Col 29
Spaces: 4
UTF-8
CRLF
JavaScript
84°F
Sunny
Search
96%
10:53 AM
10/20/2026

Fetching and Pulling from Your Remotes

As you just saw, to get data from your remote projects, you can run:

```
$ git fetch <remote>
```

The command goes out to that remote project and pulls down all the data from that remote project that you don't have yet. After you do this, you should have references to all the branches from that remote, which you can merge in or inspect at any time.

If you clone a repository, the command automatically adds that remote repository under the name "origin". So, `git fetch origin` fetches any new work that has been pushed to that server since you cloned (or last fetched from) it. It's important to note that the `git fetch` command only downloads the data to your local repository — it doesn't automatically merge it with any of your work or modify what you're currently working on. You have to merge it manually into your work when you're ready.

If your current branch is set up to track a remote branch (see the next section and Git Branching for more information), you can use the `git pull` command to automatically fetch and then merge that remote branch into your current branch. This may be an easier or more comfortable workflow for you; and by default, the `git clone` command automatically sets up your local `master` branch to track the remote `master` branch (or whatever the default branch is called) on the server you cloned from. Running `git pull` generally fetches data from the server you originally cloned from and automatically tries to merge it into the code you're currently working on.

Note	From Git version 2.27 onward, `git pull` will give a warning if the `pull.rebase` variable is not set. Git will keep warning you until you set the variable. If you want the default behavior of Git (fast-forward if possible, else create a merge commit): `git config --global pull.rebase "false"` If you want to rebase when pulling: `git config --global pull.rebase "true"`

Pushing to Your Remotes

When you have your project at a point that you want to share, you have to push it upstream. The command for this is simple: `git push <remote> <branch>`. If you want to push your `master` branch to your `origin` server (again, cloning generally sets up both of those names for you automatically), then you can run this to push any commits you've done back up to the server:

```
$ git push origin master
```

This command works only if you cloned from a server to which you have write access and if nobody has pushed in the meantime. If you and someone else clone at the same time and they push upstream and then you push upstream, your push will rightly be rejected. You'll have to fetch their work first and incorporate it into yours before you'll be allowed to push. See Git Branching for more detailed information on how to push to remote servers.

Cloning an Existing Repository

If you want to get a copy of an existing Git repository — for example, a project you'd like to contribute to — the command you need is `git clone`. If you're familiar with other VCSs such as Subversion, you'll notice that the command is "clone" and not "checkout". This is an important distinction — instead of getting just a working copy, Git receives a full copy of nearly all data that the server has. Every version of every file for the history of the project is pulled down by default when you run `git clone`. In fact, if your server disk gets corrupted, you can often use nearly any of the clones on any client to set the server back to the state it was in when it was cloned (you may lose some server-side hooks and such, but all the versioned data would be there — see Getting Git on a Server for more details).

You clone a repository with `git clone <url>`. For example, if you want to clone the Git linkable library called `libgit2`, you can do so like this:

```
$ git clone https://github.com/libgit2/libgit2
```

That creates a directory named `libgit2`, initializes a `.git` directory inside it, pulls down all the data for that repository, and checks out a working copy of the latest version. If you go into the new `libgit2` directory that was just created, you'll see the project files in there, ready to be worked on or used.

If you want to clone the repository into a directory named something other than `libgit2`, you can specify the new directory name as an additional argument:

```
$ git clone https://github.com/libgit2/libgit2 mylibgit
```

That command does the same thing as the previous one, but the target directory is called `mylibgit`.

Git has a number of different transfer protocols you can use. The previous example uses the `https://` protocol, but you may also see `git://` or `user@server:path/to/repo.git`, which uses the SSH transfer protocol. Getting Git on a Server will introduce all of the available options the server can set up to access your Git repository and the pros and cons of each.

Initializing a Repository in an Existing Directory

If you have a project directory that is currently not under version control and you want to start controlling it with Git, you first need to go to that project's directory. If you've never done this, it looks a little different depending on which system you're running:

for Linux:

```
$ cd /home/user/my_project
```

for macOS:

```
$ cd /Users/user/my_project
```

for Windows:

```
$ cd C:/Users/user/my_project
```

and type:

```
$ git init
```

This creates a new subdirectory named `.git` that contains all of your necessary repository files — a Git repository skeleton. At this point, nothing in your project is tracked yet. See Git Internals for more information about exactly what files are contained in the `.git` directory you just created.

References

Chacon, S., & Straub, B. (2013). Git Pro (2nd ed.).

GeeksforGeeks. (2024, June 12). What's The Difference Between git reset mixed, soft, and hard? GeeksforGeeks. *https://www.geeksforgeeks.org/git/whats-the-difference-between-git-reset-mixed-soft-and-hard/*